

Deep Learning Optimized - Jean Zay

Introduction – Jean Zay – GPU

INSTITUT DU DÉVELOPPEMENT ET DES RESSOURCES EN INFORMATIQUE SCIENTIFIQUE

CC BY NC SA

DLO-JZ

1st part of the IDRIS course.

Commented slides.

Authors : Bertrand Cabot, Myriam Peyrounette

July 2026

Chapters:

- Introduction to DLO-JZ
- Jean Zay
- The challenges of scalability
- GPU Computing
- Tensor Cores

DLO-JZ presentation

IDRIS ◄

Agenda ◄

Presentation of the participants ◄

The purpose of this introductory section is to describe the topics to be covered during the course, the progression of the practical exercises, and to initiate a short discussion between the participants.

The goal of this course is to present good practices in optimizing a Deep Learning training on a supercomputer, particularly on Jean Zay. We will look at the system aspects and algorithmic techniques for accelerating a training.

The subjects addressed revolve around training acceleration and large model training.

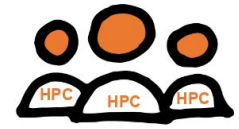
This course will not cover the state-of-the-art in Deep Learning and its different applications. We consider that you already have the necessary knowledge of the different Deep Learning techniques.

IDRIS

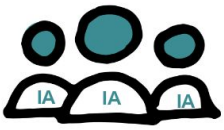
System



User Assistance



10 ingénieur·e·s



10 ingénieur·e·s

Trois équipes travaillent sur le super-calculateur:

- **Équipe système** s'occupe directement des machines (hardware, système d'exploitation, bases logicielles)
  - Membres internes à l'IDRIS
  - Ingénieur.e.s d l'entreprise constructrice (EVIDEN)
- **Équipe Support Utilisateur**
  - Équipe HPC
  - Équipe IA (formateurs DLOJZ)

BLOOM on Jean Zay

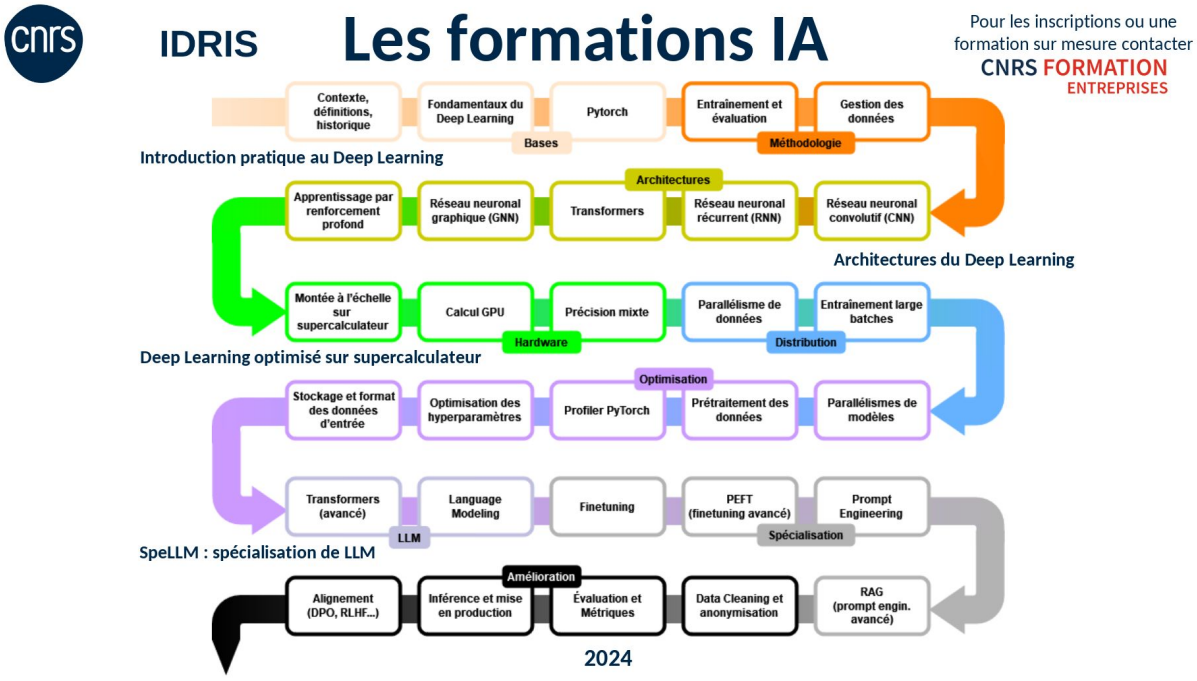


Pre-training :
117 days (2022)
384 x 80GB A100 GPUs

Palmares de Jean Zay:

- Entrainement de BLOOM en 2022 (BigScience Large Open-science Open-access Multilingual Language Model)
  - Effort de transparence dans l’entraînement de modèles d’IA

Train modulaire des formations IDRIS



==> TODO: Mettre à jour ce graphique ?

Deux formations “soeurs” à DLO:

- IPDL et ArchDL: Se familiariser avec les bases de l’IA
- SpeLLM (plus donnée pour le moment): Approfondir les connaissances et les compétences sur LLM



DLO-JZ

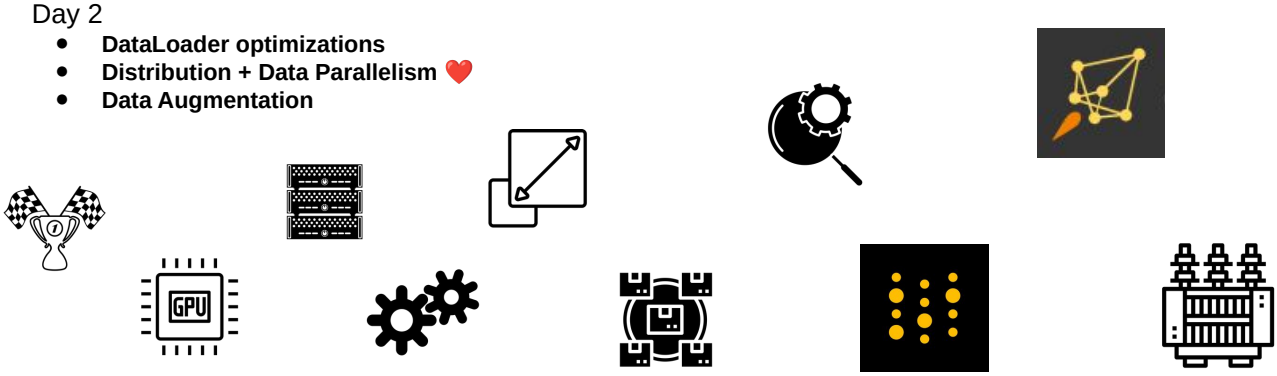
Agenda – Covered topics

Day 1
- Presentation of Jean Zay
- The challenges of scaling up
- **GPU computing + Mixed Precision**
- Tensor optimisation
- The PyTorch profiler

Day 2
- **DataLoader optimizations**
- **Distribution + Data Parallelism** ❤️
- **Data Augmentation**

Day 3
- Data storage and formats
- **Training and large batches**
- JIT

Day4
- Good practices
- Large Model Scaling + FSDP
- **Model parallelisms**
- Visualization tools
- HyperParameter Optimization



The 5 principal subjects covered during this presentation are:

- GPU computing
- Distribution on multiple GPUs in Data Parallelism
- Optimizing data preprocessing (DataLoader)
- Optimizers and their parameterization with large batches
- Model parallelisms for very large models

The first day, we will focus on problems related to the system.

The second and third days, we will look at the algorithmic optimization techniques linked to Data Parallelism and to the induced usage of large batches.

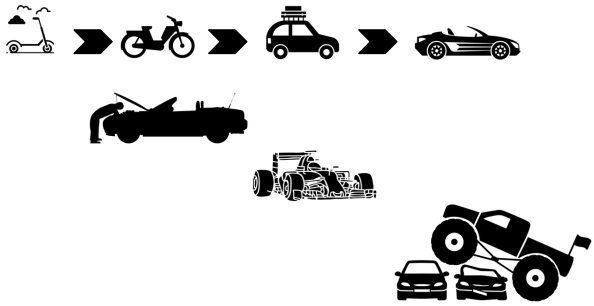
The fourth day will center on parallelism solutions for very large models (more than 1 billion parameters).

We will also look at visualization tools:

- The PyTorch Profiler for the system aspects
- Weights & Biases for the training results

Practical Workshop

- Day 1, 2, 3, 4 :
  - System Optimization : GPU, Mixed Precision
  - Profiler
  - DataLoader
  - Distributed Data Parallelism
  - Data Augmentation
  - *Optimizer*s & LR scheduler
  - torch.compile & torchscript
  - *FSDP*
  - *Hyper-Parameters Optimization (HPO)*



- Reservation partition A100 JZ

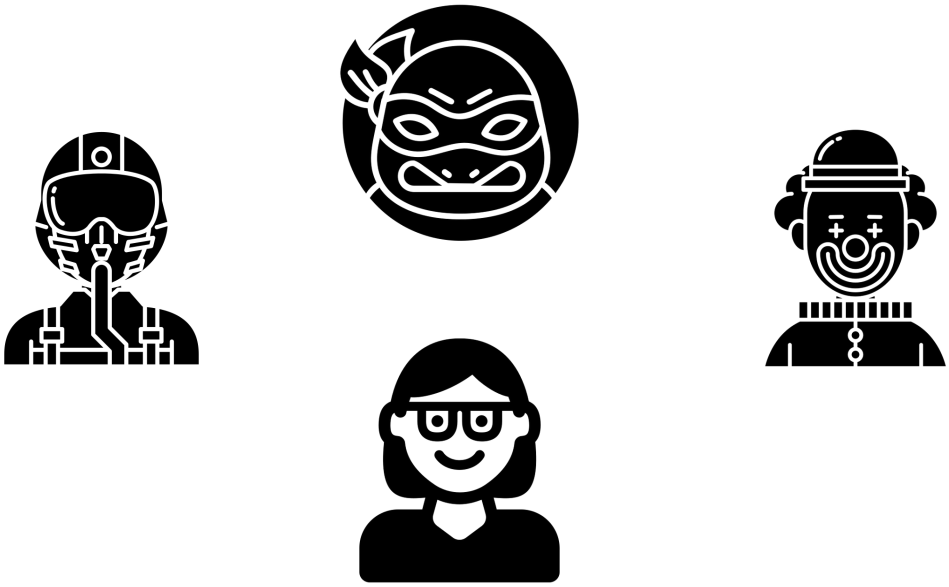


The Practice Exercises will be done exclusively in PyTorch. This choice was made because PyTorch is perfectly optimized for the Jean Zay NVIDIA GPUs.

The prerequisites for the Practice Exercises are as follows:

- Mastery of a Python code
- Knowledge of the principles of a training in Deep Learning
- The usage of PyTorch.

Presentation of participants



Once the objectives of this course have been clarified, we will invite the participants to introduce themselves and express their expectations for the course.

Jean Zay

Supercomputer ◂

Jean Zay ◂

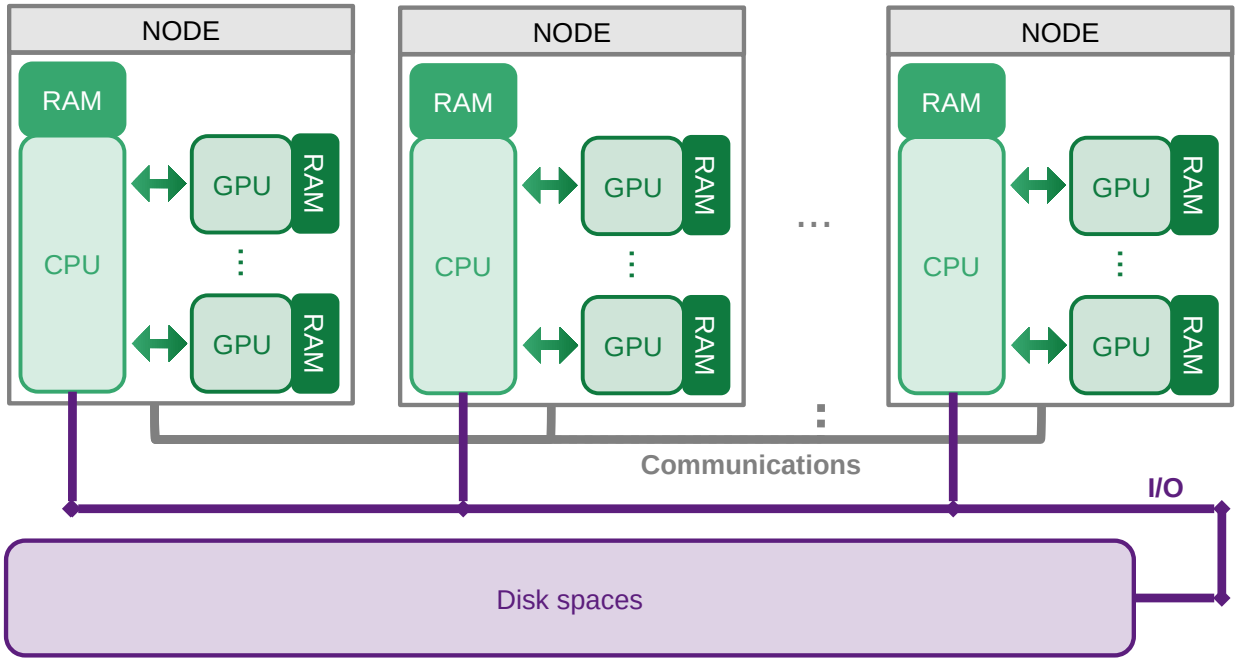
Job submission with Slurm ◂

JupyterHub on Jean Zay ◂

Slurm tools for python notebooks ◂

This section is dedicated to the Jean Zay supercomputer and will describe the computing environment of the hands-on exercises.

What’s a supercomputer?



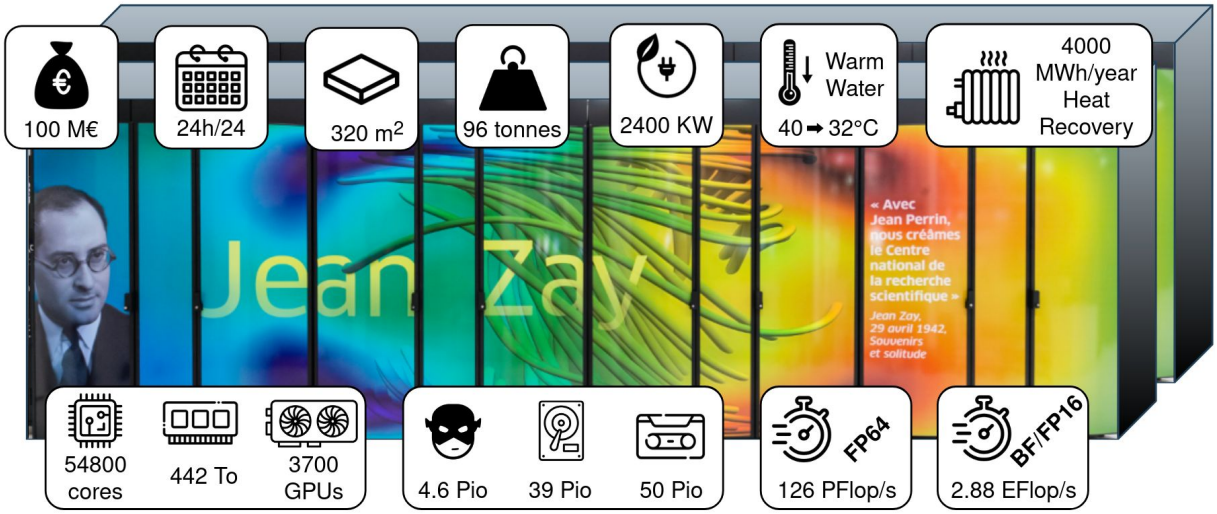
A supercomputer is an aggregation of compute nodes. Each node contains CPU cores accelerated by GPU cards, and associated RAMs.

All the supercomputer nodes together can be considered as a unique machine with aggregated performance on condition that a high performance communication network connects all the nodes (Omni-Path or InfiniBand on Jean Zay).

The storage spaces are external to the compute nodes. The files are handled by a high performance parallel file system (Lustre on Jean Zay).

Jean Zay

First national converged supercomputer dedicated to Artificial Intelligence (AI) and High Performance Computing (HPC)



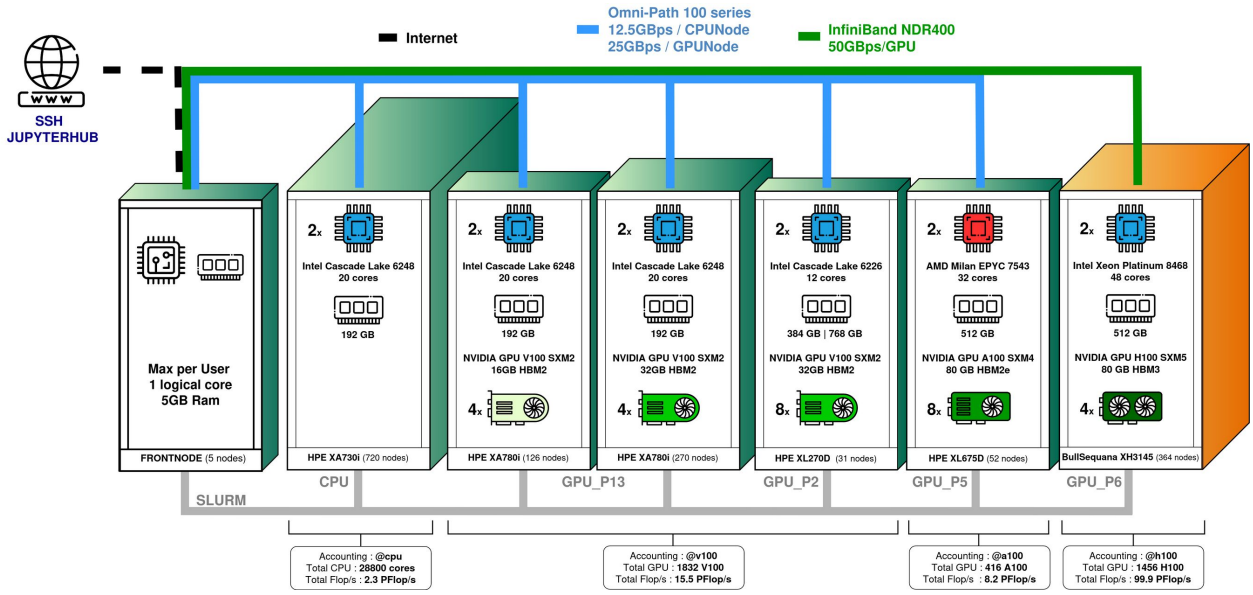
Jean Zay has been installed in 2019 and has been expanded four times since then. The current characteristics of the supercomputer are presented here.

Jean Zay is the first converged supercomputer for Artificial Intelligence (AI) and High Performance Computing (HPC) in France.

A grand scale infrastructure is necessary to host this equipment dedicated to the needs of modern research (electrical supply, cooling system, security measures,...).

The entire infrastructure requires management by a considerable number of human resources: administrative personnel and engineers (~40 employees at IDRIS).

Jean Zay

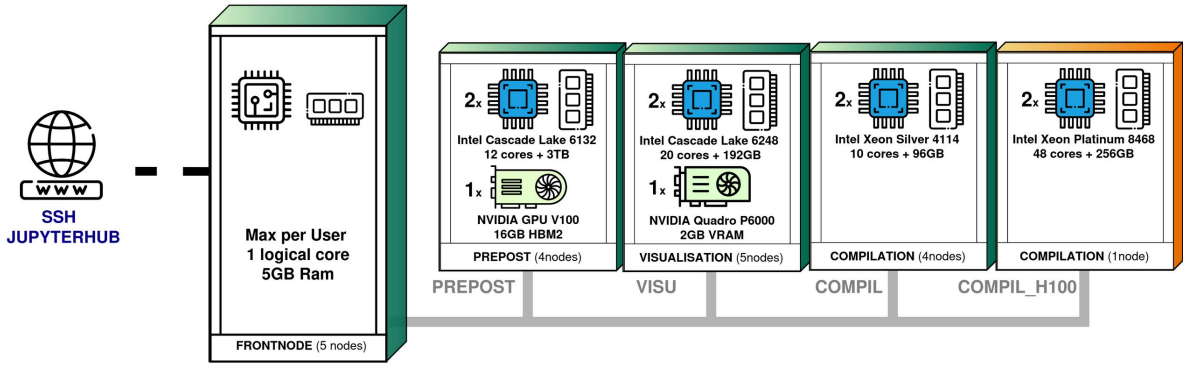


Jean Zay users can reserve compute resources on 5 different partitions having different configurations:

- A CPU partition (HPE)
- An accelerated V100 quadri-GPU partition (HPE)
- An accelerated V100 octo-GPU partition (HPE)
- An accelerated A100 octo-GPU partition (HPE)
- An accelerated H100 quadri-GPU partition (Bull)

Access to the machine is through 5 front ends which are open to the exterior (SSH and internet). Compute nodes are accessed from these front ends through the Slurm job scheduler. The compute nodes are isolated from the exterior.

Jean Zay

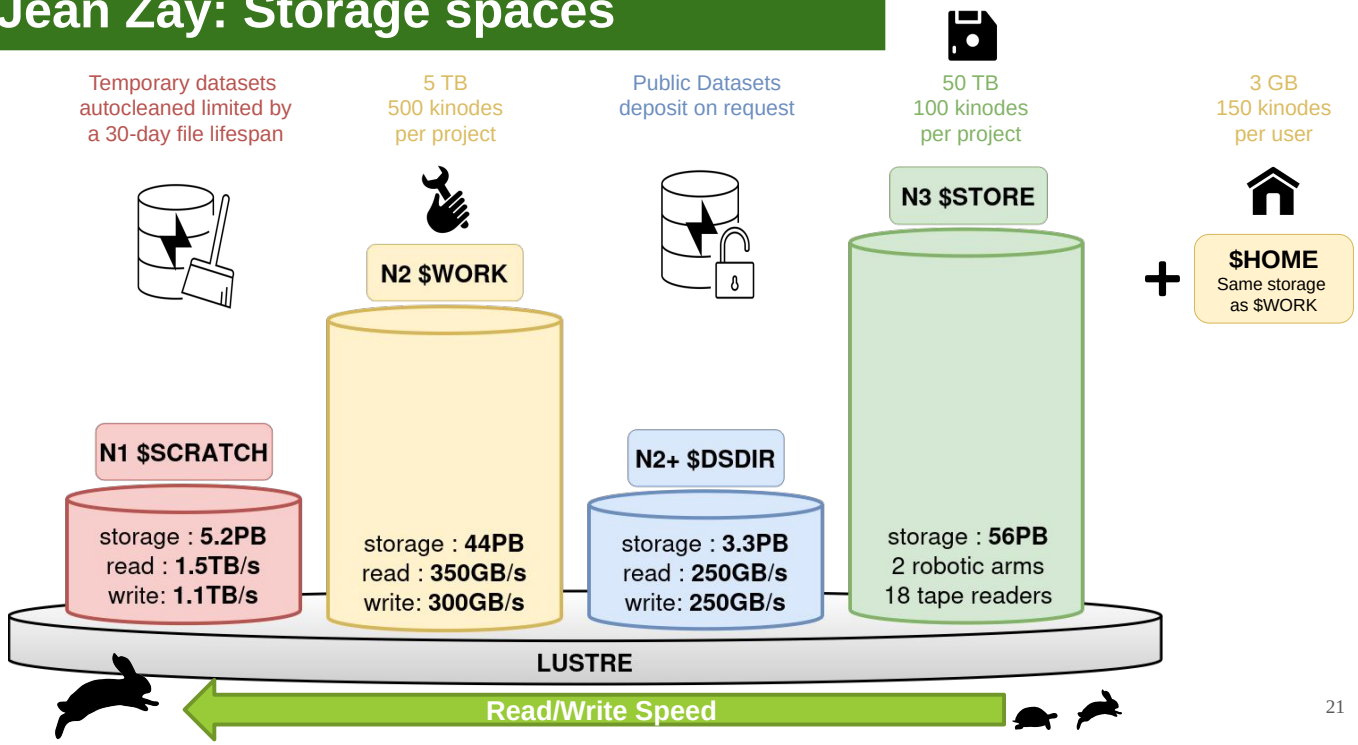


Dedicated partitions which are free of hourly computing charges and open to the exterior are also in the system:

- A “prepost” partition for all pre/postprocessing work
- A “visu” partition dedicated to visualization
- A “compil” partition dedicated to heavy compilation for codes executed on CPU, V100 and A100 nodes
- A “compil_h100” partition dedicated to heavy compilation for codes executed on H100 nodes
- An “archive” partition dedicated to long data transfers (copy or downloading). This partition is composed of nodes from the partitions listed above, thus does not appear on the figure.

These nodes are accessed from the front ends through the Slurm job scheduler.

Jean Zay: Storage spaces



Various storage disk spaces are available.

- The SCRATCH to benefit from very large storage quotas and a high read/write speed for I/O intensive jobs (Full Flash technology)
- The WORK to store code, data, logs, etc (rotative disks)
- The DSDIR to access public datasets or models (Full Flash technology, managed by the IDRIS support team and accessible by all users)
- The STORE to archive data (tapes + rotative disks)
- The HOME to store small configuration files of the computing environment (rotative disks)

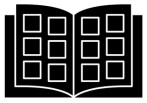
Jean Zay: Work environment



ENVIRONMENT MODULES

Catalogue of shared modules

(conda environments)



- Installed by IDRIS
- Completed on request

```
login@jean-zay3:~$ module load pytorch-gpu/py3/1.11.0
  Loading requirement: ...
(pytorch-gpu-1.11.0+py3.9.12) login@jean-zay3:~$
```

- Customizable

```
~$ pip install --user --no-cache-dir <paquet>
```



Conflicts between versions
Storage spaces saturation

Personal conda environments

```
login@jean-zay3:~$ module load anaconda-py3/2023.03
(base) login@jean-zay3:~$ conda create -n myenv
```



Storage spaces saturation ++

Singularity containers

```
login@jean-zay3:~$ module load singularity
```



Import SIF images on Jean Zay

- From your PC
- From public deposits
- Possible conversion from docker





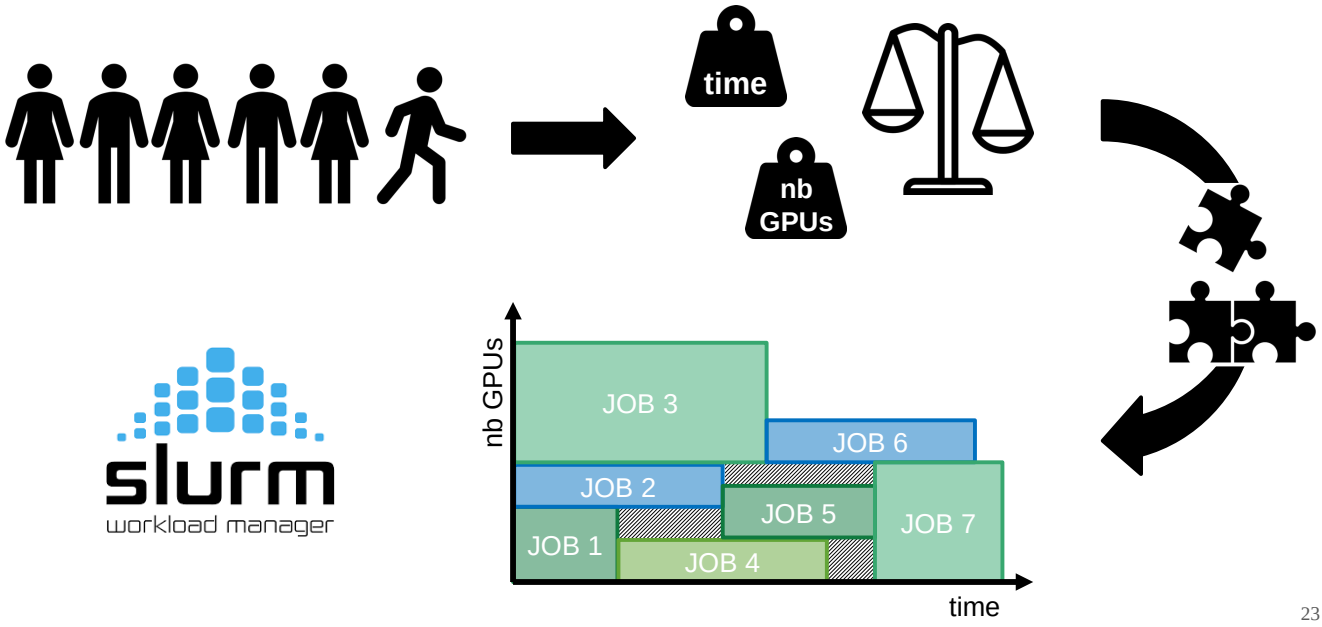
Users have a catalogue of predefined environments at their disposal. These environments are installed on Jean Zay by IDRIS. For AI users, they correspond to conda environments which are built around major libraries such as PyTorch or TensorFlow.

Computing environments are handled by the *Environment Modules v4* package.

Users just need to load the associated module to activate the desired conda environment. They are free to build or enrich their own environments. However, we recommend that you request IDRIS to directly enrich the catalogue environments to pool the resources and avoid saturating your project disk spaces.

Users can also import an environment on Jean Zay via Singularity containers.

Job submission with Slurm



The distribution of the computing resources is managed by the job scheduler Slurm.

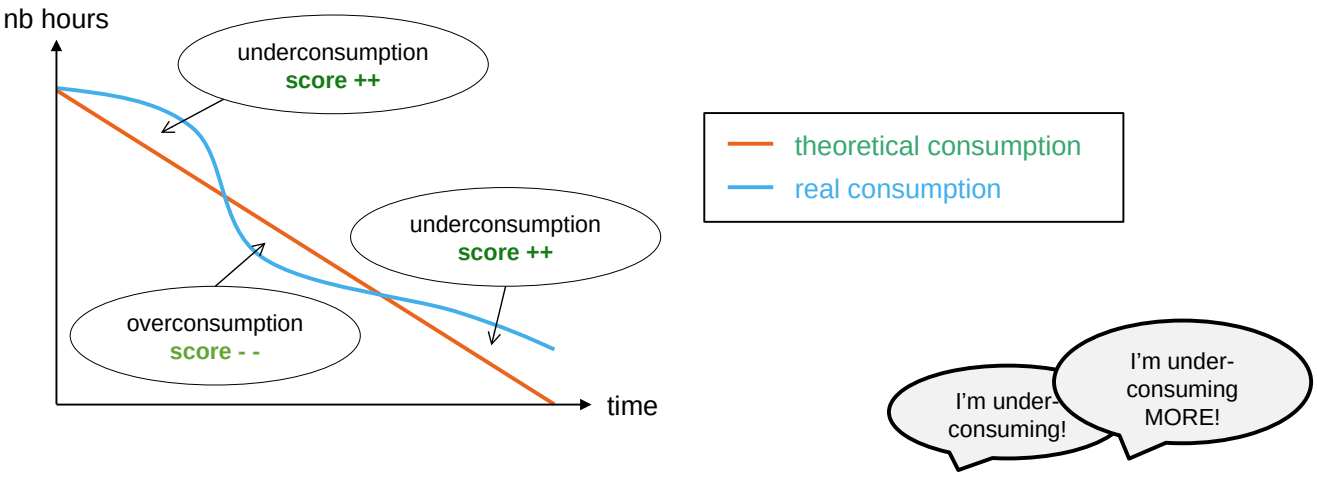
Jobs submitted by users are sent to a wait queue and Slurm plan their executions according to the requested resources (number of GPUs and execution time).

Slurm takes into account other factors such as project under/over-consumption and requested QoS.

Slurm's goal is to minimize the idle time of the computing resources, while distributing resources among users as fairly as possible.

Job submission with Slurm

Slurm gives your job a **priority score** depending on your consumption.



Slurm compares all users' scores to define job position in the queue.

A submitted job will remain in the queue until the necessary resources become available.

Slurm assigns a priority to your submitted jobs based on your past usage. This helps to ensure a linear pattern of resource consumption throughout the duration of your allocation.

Job submission with Slurm

Adjust the **QoS (Quality of Service)** to improve the priority score of your job!



QoS	Max elapsed time	Resource limits				
		Per job	Per user	Per project	Per QoS	
qos_gpu-dev	2h	32 GPUs	32 GPUs (10 jobs max at the same time)	32 GPUs	512 GPUs	dev
qos_gpu-t3 (default)	20h	512 GPUs	512 GPUs	512 GPUs		prod
qos_gpu-t4 (V100)	100h	16 GPUs	96 GPUs	96 GPUs	256 GPUs	

And keep your job in the queue, its priority score will increase with time.

The priority also depends on the chosen *Quality of Service (QoS).*

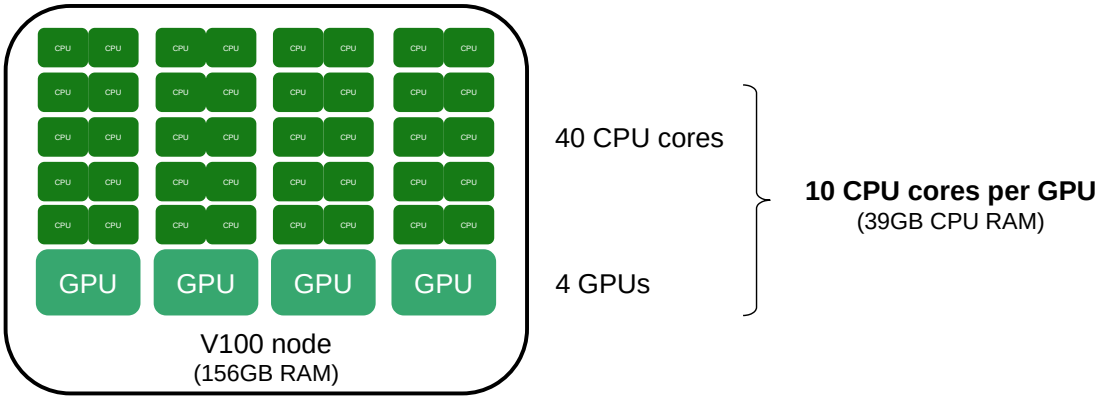
A higher-priority QoS will allow fewer resources to be reserved for a shorter maximum duration.

qos_dev is suitable for short jobs carried out for testing purposes.

Job submission with Slurm

How to configure my job?

- How many GPUs?
- How much time?
- How many CPUs per GPU? → **1 CPU core = 3.9GB RAM** (on V100 nodes)
  → interesting to have **several CPU cores to feed a GPU**
  (cf “DataLoader optimizations” part of this course)



When submitting a job, you must specify:

- The number of GPUs
- How much time your code needs to be completed
- The number of CPUs allocated to each GPU

Computing hours are calculated in proportion to the number of GPUs requested for the job, meaning that running 2 hours on 4 GPUs will cost 2 x 4 = 8 GPU hours for instance.

The number of CPUs you ask is important because:

- On Jean Zay, the amount of CPU RAM allocated to your job is proportional to the number of reserved CPUs
- By reserving several CPUs per GPU, you will be able to assign several workers to your DataLoader so it can prepare several batches at once (see the chapter about DataLoader optimisations)

On the V100 partition, it makes sense to reserve 10 CPUs per GPU since the nodes are composed of 40 CPUs and 4 GPUs.

Please note that reserving too many CPUs for a job may prevent access to the other GPUs on the node. In this case, your job will be counted as having reserved one or more additional GPUs.

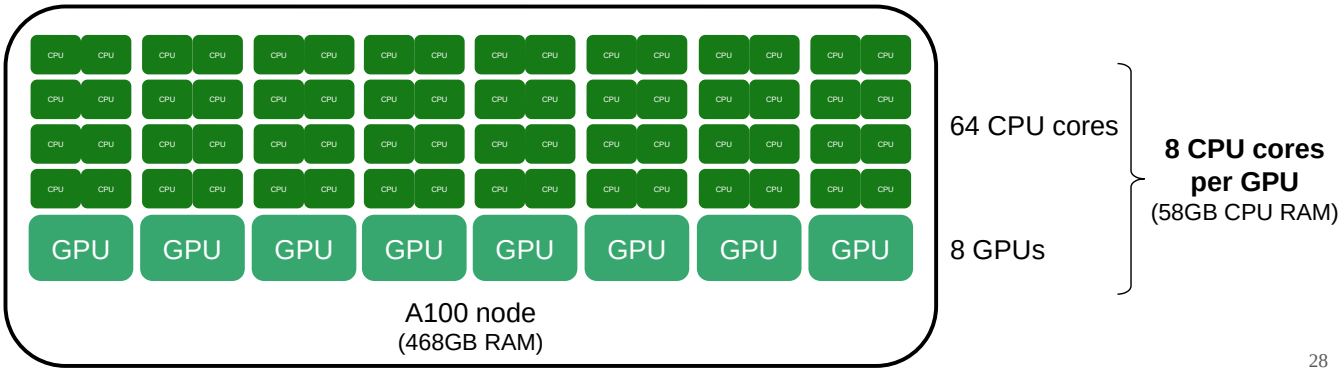
NB: Reserving more than one node will be counted as reserving the entirety of all these.

Job submission with Slurm

How to configure my job?

- How many GPUs?
- How many time?
- How many CPUs per GPU?

→ **1 CPU core = 7.3GB RAM** (on A100 nodes)
→ interesting to have **several CPU cores to feed a GPU**
(cf “DataLoader optimizations” part of this course)



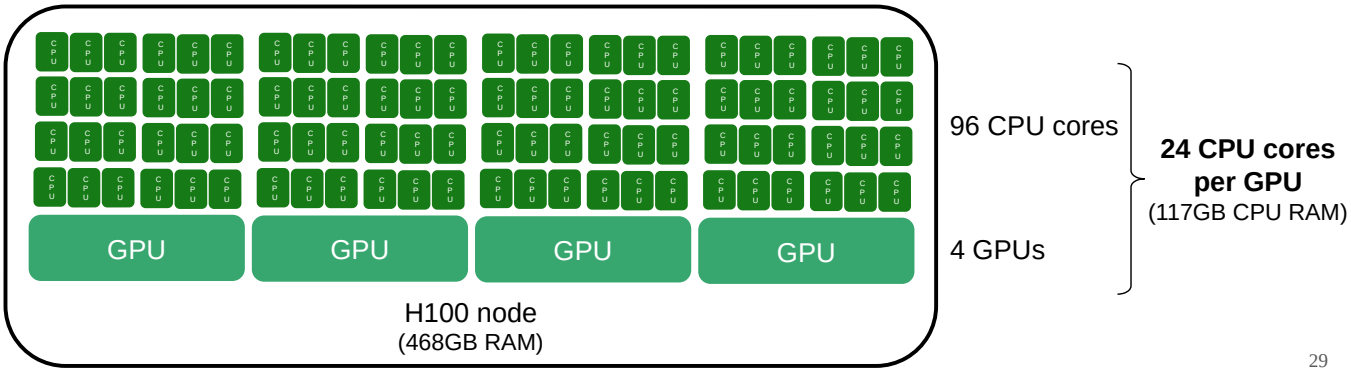
On the A100 partition, it makes sense to reserve 8 CPUs per GPU since the nodes are composed of 64 CPUs and 8 GPUs.

Job submission with Slurm

How to configure my job?

- How many GPUs?
- How many time?
- How many CPUs per GPU?

→ **1 CPU core = 4.87GB RAM** (on H100 nodes)
→ interesting to have **several CPU cores to feed a GPU**
(cf “DataLoader optimizations” part of this course)



On the H100 partition, it makes sense to reserve 24 CPUs per GPU since the nodes are composed of 96 CPUs and 4 GPUs.

Job submission with Slurm

Example: reservation of 2 x 4 V100 GPUs

script.slurm

```
#!/bin/bash

#SBATCH --job-name="dlojz"              # name of the job
#SBATCH --output="log%j.out"            # output file
#SBATCH --error="log%j.err"             # error file
#SBATCH --nodes=2                       # nb of nodes
#SBATCH --gres=gpu:4                    # nb of GPUs/node
#SBATCH --ntasks-per-node=4             # nb of tasks/node
#SBATCH --cpus-per-task=10              # nb of CPU cores/task
#SBATCH --hint=nomultithread            # no hyperthreading
#SBATCH --time=01:00:00                 # max execution time
#SBATCH --qos=qos_gpu-dev               # adjust QoS

module load pytorch-gpu/py3/2.11.0      # environment

srun python script.py                   # run script
```


To submit a job via Slurm, users need to create a batch script defining all the necessary parameters to reserve a computing slot, and the commands they want to be executed on these resources.

To launch a parallel execution on multiple tasks (or processes), the execution command of the python script must be preceded by a call to the parallel Slurm launcher *srun*.

Job submission with Slurm

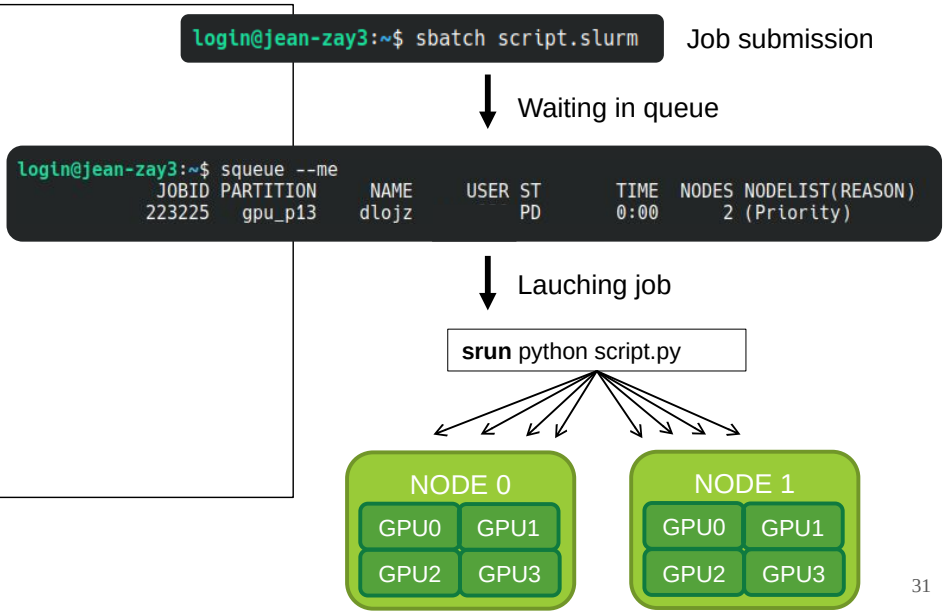
script.slurm

```
#!/bin/bash

#SBATCH --job-name="dlojz"
#SBATCH --output="log%j.out"
#SBATCH --error="log%j.err"
#SBATCH --nodes=2
#SBATCH --gres=gpu:4
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=10
#SBATCH --hint=nomultithread
#SBATCH --time=01:00:00
#SBATCH --qos=qos_gpu-dev

module load pytorch-gpu/py3/2.11.0

srun python script.py
```



Once the batch script is written, you can submit it thanks to the *sbatch* command.

Slurm will send the job in the wait queue. The jobs currently handled by Slurm can be listed with the *squeue* command.

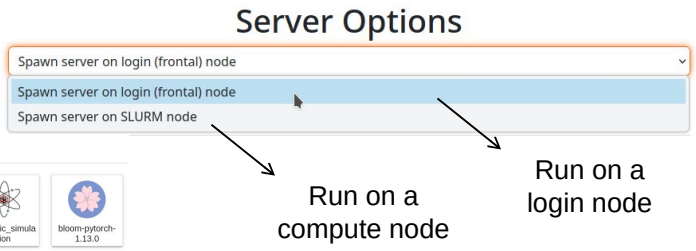
Once the computing resources are available, the execution is automatically launched by Slurm on all the requested tasks (thanks to the parallel launcher *srun* called in the batch script).

JupyterHub on Jean Zay

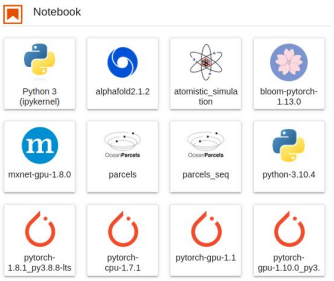
1. Authentication on https://jupyterhub.idris.fr



2. Choose and configure an instance



3. Choose a kernel (pytorch-gpu-2.11.0)



You can access Jean Zay computing resources through a JupyterHub platform: https://jupyterhub.idris.fr/.

This platform has been developed by IDRIS engineers to comply with the security constraints of the center.

You can run a Jupyter instance:

- Either on a front end, to use it like an augmented shell enabling job submission on the compute nodes, data preprocessing, visualization of results,... This front end has limited resources and is not equipped with GPUs.
- Or directly on a compute node, in order to run tests in interactive mode. In this case, access to the internet is lost. This configuration is restricted to one node.

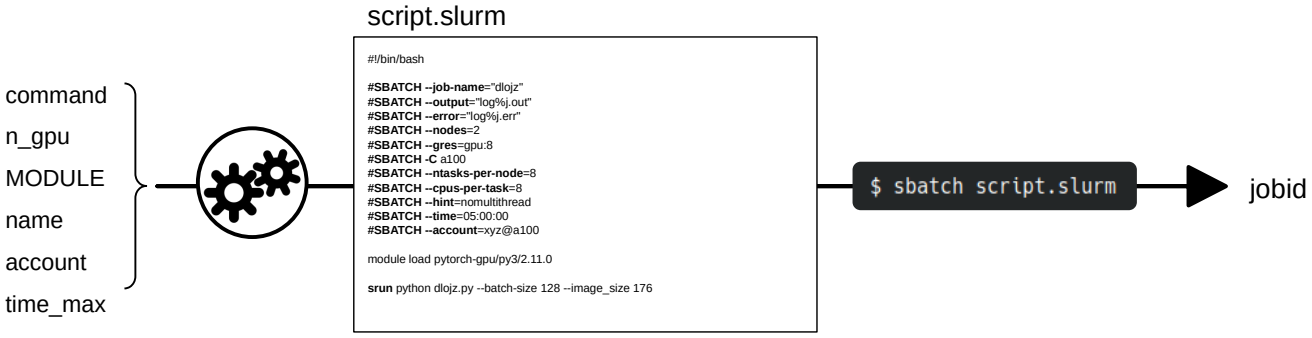
We will launch a Jupyter instance from a front end during the practice exercises of this training.

Slurm tools for python notebooks

```
from idr_pytools import gpu_jobs_submitter
```

```
command = 'dlojz.py --batch-size 128 --image_size 176'
n_gpu = 8
MODULE = 'pytorch-gpu/py3/2.11.0'
name = 'dlojz'

jobid = gpu_jobs_submitter(command, n_gpu, MODULE, name=name, account='xyz@a100', time_max='05:00:00')
```



IDRIS developed a python library called *idr_pytools* to ease the interaction with Slurm from a python notebook.

The *gpu_jobs_submitter* function allows you to easily submit jobs.

From a reduced number of instructions (the command to execute, the number of GPUs, the module to load, the name you want to give to your job,...), *gpu_jobs_submitter* will write the corresponding batch script for you and submit it.

This function will be used during the practice excercises of this training.

Slurm tools for python notebooks

```
from idr_pytools import display_slurm_queue
```

```
name = 'dlojz'
display_slurm_queue(name)
```

→ `$ squeue --me -n <name>`

```
from idr_pytools import search_log
```

```
jobid = ['12345']

search_log(contains=jobid)[0]

search_log(contains=jobid, with_err=True)[0]
```

→ *output* filename

→ *error* filename

Moreover, the *idr_pytools* library allows you to visualize the Slurm wait queue from a python notebook thanks to the *display_slurm_queue* function.

The log files generated during the execution of a job can be displayed thanks to the *search_log* function.

This function will be used during the practice exercises of this training.